Disaster Recovery Plan for a Cloud Based Azure Web Application Using Restic

Introduction

Cloud infrastructure gets talked about as inherently resilient far too often, when in reality Azure guarantees the availability of the platform itself, not the survival of whatever a customer builds on top of it. A regional outage, a compromised VM, or someone on the team running the wrong DROP statement against production are all failures that sit squarely with the customer to plan for, not the provider. Carson and Rodrick (2024, ch. 2) make this distinction clear when they separate infrastructure availability from data recoverability, arguing that a disaster recovery plan has to assume the underlying platform will occasionally fail in ways outside anyone's control, and design around that rather than around it never happening. This post sets out a DR plan for a small Azure hosted web application, describes the backup strategy built around Restic, defines the recovery objectives the plan is measured against, and then walks through an actual simulated failure to see whether those objectives were met in practice rather than just on paper.

The Architecture Being Protected

The system being protected is a two tier web application hosted on Azure, structured to mirror a realistic small production deployment rather than a toy example. An Ubuntu based Azure Virtual Machine runs the application layer, an Nginx reverse proxy in front of a Flask backend, while a second Azure VM in the same virtual network runs PostgreSQL as the persistence layer. User uploaded files sit on an attached managed disk on the application VM, and static assets are served from Azure Blob Storage. The two VMs communicate over a private subnet, with only the application tier exposed through a public IP and a network security group restricting inbound traffic to HTTP and HTTPS.

The reason this matters for a DR plan is that it defines what actually needs protecting. The database is the component most exposed to accidental deletion or corruption, since it is the one thing a single bad SQL statement can destroy in seconds. The application server itself is comparatively disposable, since it holds no state of its own beyond configuration and uploaded files, both of which can be backed up and restored onto a freshly provisioned VM if the original were lost entirely, whether through deletion, ransomware style file loss, or the VM itself becoming unrecoverable.

Backup Strategy and Tool Selection

I chose Restic over the other options listed in the brief, Bacula, Duplicity and BorgBackup, mainly because of how it fits this scale of deployment. Bacula is built around a director and storage daemon architecture that assumes a fleet of machines

being backed up centrally, which is more infrastructure than a two VM deployment needs. BorgBackup is closer to Restic in design but has weaker native support for cloud object storage backends. Restic, by contrast, has first class support for Azure Blob Storage as a repository target, along with built in encryption and block level deduplication, which keeps storage costs down on repeated daily backups of largely unchanged data (Restic, n.d.).

The backup strategy covers three things on the application VM: a PostgreSQL dump of the database, the uploaded files directory, and the application configuration. These are backed up hourly through a scheduled cron job, with each Restic snapshot pushed to a dedicated Azure Storage Account in a separate Azure region from the primary deployment, so that a regional disruption affecting the production VMs would not also take out the backup repository.

bash

```
# Restic repository backed by Azure Blob Storage in a separate region
export RESTIC_REPOSITORY="azure:shopease-dr-backups:/restic-repo"
export AZURE_ACCOUNT_NAME="shopeasedrstorage"
export AZURE_ACCOUNT_KEY="<account-key>"
export RESTIC_PASSWORD="<repository-password>"

restic init

# Hourly backup job
pg_dump -U shopease_admin -d shopease > /tmp/shopease_db.sql
restic backup /tmp/shopease_db.sql /var/www/shopease/uploads /etc/shopease
```

Retention is managed through Restic's forget and prune commands, keeping daily snapshots for a week and weekly snapshots for a month, which stops the repository growing indefinitely while still giving a reasonable window to recover from a problem that was not noticed immediately.

bash

```
restic forget --keep-daily 7 --keep-weekly 4 --prune
```

Recovery Objectives

Carson and Rodrick (2024, ch. 2) define Recovery Time Objective as the maximum acceptable length of time a system can be down before the disruption becomes unacceptable to the business, and Recovery Point Objective as the maximum acceptable amount of data loss, measured as a window of time back from the point of failure rather than a data volume. Both figures are meant to drive the design of a backup strategy rather than being decided after the fact, since an RPO of fifteen minutes implies backups need to run roughly that often, while an RTO of a few minutes implies the restore process itself needs to be close to instant, which usually rules out a full restore from cold storage.

For this deployment I set an RTO target of thirty minutes and an RPO target of sixty minutes. Neither is especially aggressive, which is deliberate. A small application like this does not carry the same continuous transaction volume as a banking platform, so an hour's worth of backed up data lost in the worst case is a reasonable trade off against the cost and complexity of a shorter backup interval.

Simulating the Failure

To test the plan against something concrete rather than theoretical, I simulated an accidental data loss scenario on the database tier. At 09:00 the scheduled backup ran normally, producing a snapshot containing a full pg_dump of the shopease database. At 09:40 I simulated an accidental deletion by dropping a core table directly on the production database, the kind of mistake that happens during a rushed manual fix far more often than anyone would like to admit.

bash
```
sudo -u postgres psql -d shopease -c "DROP TABLE orders;"
```

At this point the application would be visibly broken, any request touching order history would fail, which is what triggered the recovery process.

Recovery Process

The first step was confirming the most recent usable snapshot existed in the Restic repository.

bash
```
restic snapshots
```

With the correct snapshot identified, the recovery itself involved restoring the backed up files to a temporary location and reloading the database dump.

bash
```
restic restore latest --target /tmp/restore
psql -U shopease_admin -d shopease < /tmp/restore/tmp/shopease_db.sql
```

The entire process, from confirming the failure to the database being fully restored and the application responding correctly again, took just under eight minutes. That included the time spent locating the correct snapshot, running the restore, and manually verifying the orders table had returned with its data intact.

Evaluation Against RTO and RPO

Against the thirty minute RTO target, the actual recovery time of roughly eight minutes comfortably met the objective, with a reasonable margin to spare even accounting for the fact that this was a controlled test rather than a real incident with the added overhead of diagnosing what had actually gone wrong first. Against the

sixty minute RPO target, the last successful backup before the simulated failure was taken at 09:00, and the failure occurred at 09:40, meaning the actual data loss window was forty minutes. This also met the target, though with less margin than the RTO result, and it highlights a genuine limitation of an hourly backup schedule, since a failure occurring at 09:59 would have come close to breaching the RPO target entirely. For a system with a lower tolerance for data loss, this would be the first thing I would revisit, either by shortening the backup interval or by supplementing the hourly full dump with continuous write ahead log shipping so that point in time recovery becomes possible rather than being limited to the granularity of the last scheduled snapshot.

Discussion

It is worth being honest about what this plan does and does not cover. Restic protects data that has already been backed up and pushed to a separate region, which is exactly what is needed against accidental deletion, database corruption, or a ransomware style compromise of the VM itself, since an attacker with access to the production VM has no way to reach into or delete snapshots already sitting in a separate storage account under different credentials. What it does not replace is Azure's own platform level resilience tooling. Azure Backup and Azure Site Recovery operate at the infrastructure level, handling whole VM replication and failover in a way that a file and database level tool like Restic was never designed to do (Microsoft, n.d.). A more complete DR posture for a production system would likely combine both, Azure Site Recovery for infrastructure level failover in the event of a regional outage, and Restic style application aware backups for the kind of logical, human caused failures that infrastructure replication alone does not protect against, since replicating a database immediately after someone drops a table simply replicates the empty table along with it.

Conclusion

This exercise showed that a fairly lightweight, open source backup tool can meet realistic recovery objectives for a small cloud deployment without needing the operational overhead of a larger backup platform. The simulated recovery met both the RTO and RPO targets set at the outset, though the margin on the RPO result in particular suggests the backup interval is the part of this plan most worth tightening if the application's tolerance for data loss were ever to become stricter than it currently is.

References

Carson, R. and Rodrick, M. (2024) *Business Continuity and Disaster Recovery*. Birmingham: Packt Publishing.

Microsoft (n.d.) *Azure Backup documentation*. Available at: https://learn.microsoft.com/en-us/azure/backup/ (Accessed: 20 September 2026).

Restic (n.d.) *Restic documentation*. Available at: https://restic.readthedocs.io/ (Accessed: 20 September 2026).